Relatório — LAB 6: Árvores B+

Questão 1 • Construção e impressão de uma árvore B+ de ordem 2 em C

Nome dos integrantes: Fabiano Carvalho Costa Junior 2520144 e Guilherme Vilas Boas Barbosa 2411726

1. Introdução

O objetivo da primeira questão foi construir, em C, a árvore B+ de ordem 2 apresentada no laboratório. Nessa configuração, cada nó pode ter até quatro chaves e cinco ponteiros para outros nós.

Os nós internos servem para orientar a busca, enquanto as folhas guardam as chaves. Como as folhas também são encadeadas, é possível percorrê-las na ordem dos valores.

2. Desenvolvimento

Começamos criando a struct dos nós. Nela, utilizamos ndesc para indicar quantas chaves estão ocupadas, um vetor chave para guardar os valores e um vetor ramo com os ponteiros para os filhos. Também colocamos o ponteiro prox para conectar uma folha à seguinte.

A função cria_no usa malloc para reservar memória e inicializa os ponteiros com NULL. Na versão atual, ela também preenche inicialmente todas as posições do vetor de chaves com -1, facilitando a identificação dos espaços vazios na impressão.

Seguindo o enunciado, criamos primeiro as seis folhas, depois os dois nós internos e, por último, a raiz. O nó interno esquerdo ficou com as chaves 50 e 75; o direito, com 120 e 200; e a raiz, com 100. Os ponteiros ramo foram ligados aos filhos correspondentes.

As folhas foram conectadas na sequência a → b → c → d → e → f. Dessa forma, a árvore ficou organizada tanto para uma busca pelos índices quanto para um percurso pelas folhas.

3. Atualização da impressão

Para facilitar a leitura, cada nó recebeu um nome simbólico (r, x, y, a, b, c, d, e e f). Essas letras representam os nós na tabela; não são os endereços reais de memória. Também aumentamos o vetor para chave[MAX + 1], deixando a quinta posição reservada para overflow, e inicializamos as posições ainda não usadas com -1.

Nó	Ponteiros ramo	Chaves (com overflow)
r	`x, y, NULL, NULL, NULL`	`100, -1, -1, -1, -1`
x	`a, b, c, NULL, NULL`	`50, 75, -1, -1, -1`
y	`d, e, f, NULL, NULL`	`120, 200, -1, -1, -1`
a	`NULL, NULL, NULL, NULL, NULL`	`10, 40, -1, -1, -1`
b	`NULL, NULL, NULL, NULL, NULL`	`50, 60, 70, -1, -1`
c	`NULL, NULL, NULL, NULL, NULL`	`75, 80, 90, -1, -1`
d	`NULL, NULL, NULL, NULL, NULL`	`100, 110, 115, -1, -1`
e	`NULL, NULL, NULL, NULL, NULL`	`120, 135, 140, 170, -1`
f	`NULL, NULL, NULL, NULL, NULL`	`200, 230, 240, 250, -1`

Os ponteiros prox continuam ligando as folhas, mas não aparecem nessa tabela, pois ela mostra apenas os cinco ponteiros ramo de cada nó.

4. Testes e resultados

A impressão agora permite conferir o formato da árvore de uma só vez. Por exemplo, na linha do nó x aparecem as chaves 50 e 75 e os ponteiros para a, b e c. Nas folhas, os ponteiros ramo são NULL, porque elas não possuem filhos.

Na execução anterior com o GDB, o programa terminou com código 0, sem sinalização de erro. Para a versão com a nova tabela, o resultado esperado é o apresentado acima; a verificação final deve ser feita executando o arquivo atualizado no terminal.

Nesta versão não colocamos chamadas a free, conforme a escolha feita para o exercício. Como o programa termina logo depois da impressão, o sistema normalmente recupera a memória do processo; em programas maiores, seria importante liberá-la explicitamente.

5. Conclusão

A atividade ajudou a entender como uma árvore B+ é montada em C e como seus ponteiros organizam os nós internos e as folhas. A nova impressão também tornou mais fácil conferir a estrutura, pois mostra as ligações entre os nós e as posições ainda vazias.

6. Desafios encontrados

Durante o desenvolvimento do laboratório, uma das principais dificuldades foi entender como organizar os nós da árvore B+ e conectar corretamente os ponteiros. Como cada nó pode ter várias chaves e filhos, foi necessário prestar atenção na posição de cada elemento para manter a estrutura igual à apresentada no enunciado.

Outro desafio foi fazer a impressão da árvore no formato mostrado pelo professor. Inicialmente, o programa imprimia apenas as chaves das folhas, mas depois foi necessário adaptar o código para mostrar também os endereços simbólicos dos nós, os ponteiros e todas as posições do vetor de chaves. Para isso, utilizamos `NULL` nos ponteiros não utilizados e `-1` nas posições vazias das chaves, incluindo a posição reservada para overflow.

No final, conseguimos organizar melhor o código e deixar a impressão mais próxima do exemplo apresentado em sala, facilitando a visualização dos nós e de suas conexões.

7. Como compilar e executar o projeto

Para compilar e executar o programa, é necessário ter o compilador GCC instalado no computador.

Após baixar o arquivo `ex1.c`, abra o VS Code e coloque o arquivo em uma pasta de sua preferência.

Em seguida, abra o terminal do VS Code, acessando **Terminal → Novo Terminal**. Certifique-se de que o terminal esteja na mesma pasta onde o arquivo `ex1.c` foi salvo.

Para compilar, execute o seguinte comando:

```
gcc ex1.c -o ex1
```

Esse comando compila o código-fonte e gera um arquivo executável chamado `ex1`.

Depois que a compilação terminar, execute o programa.

No Linux ou macOS:

```
./ex1
```

No Windows, utilizando GCC (MinGW):

```
.\ex1.exe
```

Após a execução, o programa mostrará uma tabela com os endereços simbólicos dos nós, os ponteiros e as chaves da árvore B+.

Caso apareça a mensagem de que o comando gcc não foi encontrado, será necessário instalar e configurar o compilador GCC antes de executar o programa.